WHITE PAPER

Automating SoC Reference Manual Generation

An AI-Augmented, RTL-Driven Documentation Pipeline for Semiconductor Products

Author: Jayant Kaushik · Principal Technical Writer, NXP Semiconductors

June 2025

This white paper presents a novel approach to SoC Reference Manual generation that integrates RTL design analysis via Verific, a hierarchical DITA-based content architecture, automated parameter-driven spec assembly, generative AI chapter authoring, and a RAG-powered natural language knowledge base. It is intended for documentation engineering leads, EDA tool integrators, and technical publishing architects in the semiconductor industry.

Abstract

SoC Reference Manual (RM) authoring is one of the most resource-intensive documentation activities in semiconductor product development. As SoC complexity grows — with modern devices integrating 80–150 distinct IP blocks, each parameterizable across product tiers — the traditional workflow of manually extracting design parameters, assembling XML content structures, and authoring architectural chapters is no longer sustainable at the pace of modern tape-out schedules.

This white paper describes a production-deployed platform at NXP Semiconductors that automates the end-to-end Reference Manual generation workflow. The platform combines Verific-based RTL elaboration for accurate parameter extraction, a hierarchical DITA content architecture with proprietary parameter-driven spec assembly, a JSON-based non-RTL parameter database, Claude-powered generative chapter authoring, DITA-OT parameter-resolved publishing with automated register validation, and a browser-based RM Explorer featuring a RAG-powered natural language chatbot. The result is a reduction in RM generation time from weeks to hours, with measurably improved accuracy and consistency.

1. The SoC Documentation Problem

1.1 Scale and Complexity

A contemporary NXP SoC integrates dozens to over a hundred distinct IP blocks — FlexCAN controllers, SPI interfaces, UART peripherals, audio subsystems, USB controllers, PCIe bridges, DDR memory controllers, cryptographic accelerators, and many others. Each IP block is parameterizable: its register count, feature set, clock domain, and memory footprint vary by chip tier and configuration. Each instance appears at a unique base address in the SoC memory map, with parameter overrides propagated from the chip top level through a deep RTL hierarchy.

The resulting Reference Manual must accurately document every register, every bit field, every address, and every parameter-dependent configuration for every instance — while simultaneously reusing IP-level documentation across multiple chip variants without manual duplication. At tape-out scale, this represents a documentation surface of tens of thousands of pages across the product family.

1.2 The Manual Workflow and Its Costs

In the legacy workflow, technical writers and IP architects faced the following recurring manual tasks at each tape-out cycle:

- **Parameter extraction:** Manually querying RTL design databases or soliciting current parameter values from design engineers — a process vulnerable to version skew, communication lag, and transcription error.
- **Spreadsheet parameter management:** Maintaining per-chip spreadsheets of documentation-specific (non-RTL) parameters with no schema enforcement, no versioning, and no traceability to spec content.
- **Manual spec assembly:** Hand-crafting XML content instance files, DITA map references, and publishing manifest files — time-consuming, inconsistent, and brittle to design changes.
- **Repetitive chapter authoring:** Writing architectural overview chapters (clocking, memory map, interrupt tables) from scratch for each chip — structurally similar across products but requiring individual attention.
- **No register validation:** Discrepancies between RTL register definitions and documented register specifications were not caught until post-silicon review, contributing to errata.
- **Static PDF navigation:** Engineers accessing large RMs for specific register values or parameter information had no recourse beyond full-text PDF search — inefficient for complex, multi-thousand-page documents.

The cumulative cost: a full SoC RM generation cycle required 3–4 weeks of effort, with significant quality risk introduced at each manual step.

2. Approach: An Integrated Automation Pipeline

The platform addresses each pain point with a targeted automation stage. Stages are orchestrated through a VSCode plugin that exposes a command palette and sidebar panel, backed by a local Python RPC server handling computation. The complete pipeline is shown below.

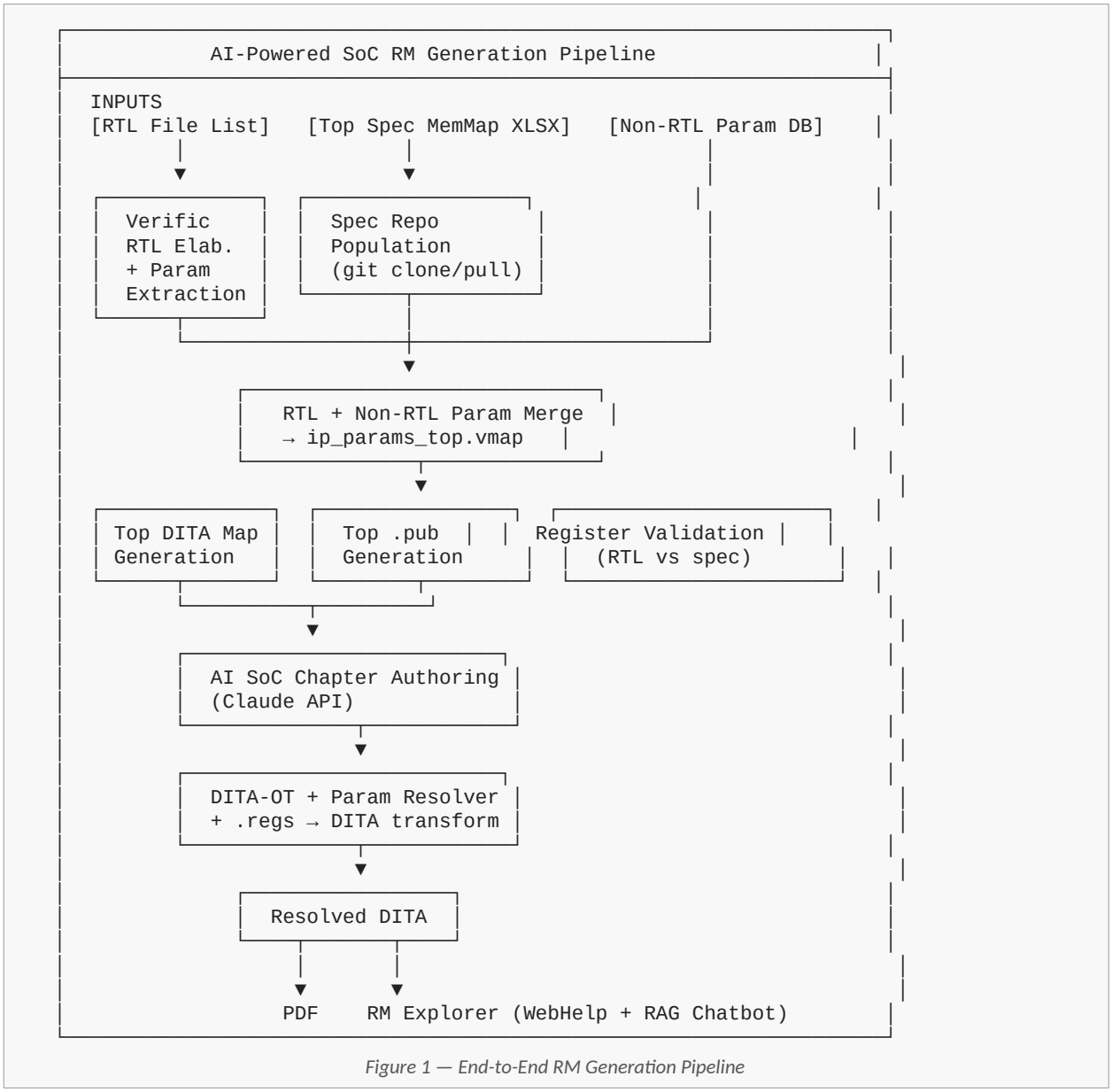

Figure 1 — End-to-End RM Generation Pipeline

3. Key Technical Components

3.1 Hierarchical Spec Content Model

The platform operates on a three-tier content hierarchy. Each tier has a standard directory structure: maps (DITA maps), topics (DITA topics), inst (content instance files), params (parameter definition files), regs (register description files), and images. Subsystem and SoC specs reference IP-level specs through their maps and instance files, enabling full single-source reuse.

Tier	Role
IP Spec (Tier 1)	Self-contained spec for a single IP block. Publishable standalone or as part of a higher-tier spec. Contains reusable register descriptions, parameterized topics, and a complete parameter definition file.
Subsystem Spec (Tier 2)	Aggregates multiple IP specs under a subsystem (e.g., Audio subsystem). Defines subsystem-level parameters not present in individual IP specs. Links to IP spec content via DITA map and publishing manifest references.
SoC Spec (Tier 3)	Top-level spec for the full chip. Auto-generated from the merged content instance file and memory map. Contains AI-authored architectural chapters plus references to all subsystem and IP specs.

3.2 Custom File Types

Four proprietary file types extend the standard DITA model to support parameter-driven authoring and publishing:

File Type	Purpose
.prm	Parameter definition file. Declares all parameters for a spec tier — data type (boolean, integer, float, string, enumerated), default value, allowed range, and RTL vs. non-RTL source classification. A global .prm covers shared publishing parameters.
.vmap	Content instance file. Binds resolved parameter values to the declared parameters. Contains Register Instance elements, each linking a register description file and carrying per-instance parameter bindings. Multiple Register Instances can reference the same .regs file with different bindings.
.pub	Spec publishing manifest. Defines instance selectors and hierarchical resolve groups governing how register information is merged and deduplicated across multiple IP instances during publishing. Subsystem and SoC manifests reference IP-level manifests.
.regs	Register description file. Proprietary XML format (analogous to IPXACT) storing complete register definitions: register name, offset, reset value, access type, and full bit-field specifications. Supports parameterized fields and conditional register blocks.

3.3 Verific-Based RTL Parameter Extraction

Accurate, automated parameter extraction is the cornerstone of the pipeline. The platform uses Verific Design Automation's SystemVerilog/VHDL parser and RTL elaborator (verific.com) to perform full static elaboration of the design hierarchy. After elaboration, all parameter overrides at every level — from chip top through subsystem to IP instance — are fully resolved, producing a module_parameters.json file with authoritative parameter values for each IP instance.

This replaces the previous LEC-based approach, which was designed for logic equivalence checking rather than parameter extraction and produced inaccurate results for deeply nested parameter overrides. Verific's elaboration accurately handles defparam propagation, generate statement unrolling, and library resolution — all common sources of error in the legacy approach.

3.4 Non-RTL Parameter Database

Documentation-specific parameters — IP revision identifiers, feature flags used only in the spec layer, chip-tier labels, protocol variant names — are not present in the RTL and cannot be extracted by design analysis. These were previously maintained in unstructured per-chip spreadsheets. The platform replaces this with a hierarchical JSON database, version-controlled alongside documentation source, recording parameter-to-instance associations and chip-prefix-keyed value overrides. The database is merged with the Verific-extracted RTL parameters during the content instance file generation step.

3.5 Generative AI Chapter Authoring

SoC-level architectural chapters follow predictable structures but require chip-specific content drawn from architecture specification documents. A Claude-based agent (using the Anthropic claude-sonnet model) receives the relevant architecture spec section, a DITA structural template, and the merged parameter context. It outputs DITA-conformant XML topics: concept topics for architectural narratives and reference topics for tabular data such as interrupt matrices and memory map overviews. The parameter context ensures that instance names, addresses, and feature flags in generated content are accurate.

3.6 Parameter-Resolved DITA Publishing

All DITA topics, maps, and register files support a custom applicability attribute enabling any content element to be conditionalized on instantiated parameter values — both RTL-sourced and non-RTL. A Python wrapper around DITA-OT runs a resolution preprocessing pass that evaluates these conditions against the active content instance file and produces fully resolved, parameter-free DITA. Register description files are simultaneously transformed into DITA reference topics with auto-generated bit-field diagrams and structured register tables.

3.7 Register Specification Validation

Verific elaboration produces register definitions alongside parameter values. These are cross-validated against the .regs files in each IP spec — checking register existence, reset values, bit-field positions, access types, and field names. Discrepancies are surfaced as structured validation reports in the VSCode plugin, enabling writers and design engineers to resolve documentation-versus-RTL inconsistencies before tape-out.

3.8 RM Explorer and RAG Chatbot

The resolved DITA output is served through a Flask-backed browser application that renders it as navigable WebHelp with register diagrams, memory map tables, and a bookmark-based table of contents. A RAG engine builds a semantic knowledge base from the resolved DITA content using sentence-transformers and FAISS indexing. A Claude-powered chatbot uses top-k retrieved context chunks to answer natural language queries — register definitions, parameter values, memory addresses, architectural relationships — directly in the browser interface.

4. Results and Impact

4.1 Quantitative Outcomes

Metric	Outcome
RM generation time	Reduced from 3–4 weeks per tape-out to under one working day for full SoC spec assembly
Parameter extraction	100% automated via Verific elaboration — zero manual RTL database queries required
Register validation	Automated cross-check covering reset values, bit ranges, access types, and field names across all IP specs — a check previously not performed at all
Query response time	Natural language RM queries answered in under 3 seconds via RAG retrieval + Claude API, vs. 15–30 minutes for equivalent PDF navigation
Param DB coverage	Non-RTL parameter associations for 100% of IP specs migrated from unstructured spreadsheets to versioned, schema-governed JSON database

4.2 Qualitative Impact

- Single-source IP spec reuse across multiple SoC products without modification — authors write once, the pipeline resolves chip-specific values at publication time
- Documentation accuracy significantly improved: Verific's RTL elaboration produces authoritative parameter values that do not drift from the design database
- Post-tape-out errata from register documentation errors reduced by automated spec validation against RTL source
- Time freed from repetitive spec assembly can be redirected to higher-value IP-level technical writing, review, and usability improvement
- Natural language RM access via the chatbot lowers the expertise barrier for hardware engineers accessing register information

5. Applicability and Generalization

While developed for NXP's specific toolchain and documentation architecture, the principles of this platform are broadly applicable to any semiconductor organization producing IP or SoC documentation at scale:

- **RTL elaboration for parameter extraction:** Any organization using SystemVerilog or VHDL can use Verific's elaboration API to produce accurate, hierarchy-resolved parameter values. The approach is design-tool-agnostic — Verific handles mixed SV/VHDL, defparam propagation, and generate statements across all major design styles.
- **Hierarchical DITA content architecture:** The IP/subsystem/SoC spec tier model is a natural fit for any organization with reusable IP documentation. The .prm/.vmap/.pub file type model can be implemented as DITA specializations or as preprocessing-layer additions to any DITA-OT pipeline.
- **Non-RTL parameter database:** The JSON-based parameter association model is straightforward to implement and adapt to any set of documentation-specific parameters that supplement RTL-derived data.
- **Generative AI chapter authoring:** The Claude-based agent approach is generalizable to any documentation type with predictable structure and architecture-spec source material. Prompt engineering and DITA template governance are the key variables.
- **RAG-powered RM querying:** The FAISS + sentence-transformers + Claude API stack is straightforward to adapt to any resolved DITA or structured XML documentation corpus.

6. Conclusion

The AI-Powered SoC Reference Manual Generation Platform demonstrates that large-scale semiconductor documentation does not need to be a bottleneck in the chip development process. By treating documentation generation as a software engineering problem — with well-defined inputs (RTL file list, memory map, parameter database), a structured content model (hierarchical DITA with parameter-driven publishing), and AI augmentation at the stages that benefit most (parameter extraction, spec assembly, chapter authoring, knowledge retrieval) — it is possible to reduce RM generation time from weeks to hours while simultaneously improving accuracy and consistency.

The platform is production-deployed at NXP Semiconductors and has been used across multiple SoC tape-out cycles. The combination of Verific-based RTL elaboration, single-source DITA parameter architecture, and Claude-powered generation and querying represents a replicable model for documentation automation in the broader semiconductor industry.

Jayant Kaushik · Principal Technical Writer, NXP Semiconductors
jayant.kaush@gmail.com · github.com/kaushikjayant/techdocs · logidocs.wordpress.com